

Q. 1 Create an HTML page to accept two numbers and write a servlet to add given numbers and display the result.

1. Project Setup in Eclipse

Step 1: Create Dynamic Web Project

1. Open Eclipse
2. Go to **File** → **New** → **Dynamic Web Project**
3. Project Name: AdditionServletApp
4. Target Runtime: Select Apache Tomcat (configure if not added)
5. Click **Finish**

Step 2: Create HTML File

1. Right click project → **New** → **HTML File**
2. Name: index.html

✧HTML Code[index.html]

```
<!DOCTYPE html>
<html>
<head>
<title>Addition Form</title>
</head>

<body>
<h2>Add Two Numbers</h2>

<form action="add" method="post">
Enter Number 1: <input type="text"
name="num1"><br><br>
Enter Number 2: <input type="text"
name="num2"><br><br>
<input type="submit" value="Add">
</form>
</body>
</html>
```

Step 3: Create Servlet Class

1. Right click project → **New** → **Servlet**
2. Class Name: AddServlet
3. Package: com.example
4. Click **Finish**

✧✧Servlet Code (AddServlet.java)

```
package com.example;

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.*;

@WebServlet("/add")
public class AddServlet extends HttpServlet
{

    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws IOException
    {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        int num1 =
            Integer.parseInt(request.getParameter("num1"));
        int num2 =
            Integer.parseInt(request.getParameter("num2"));

        int result = num1 + num2;

        out.println("<h2>Result: " + result + "</h2>");
    }
}
```

4. Configure Server (Tomcat)

1. Go to **Servers** tab
2. Click “**No servers available**” → **Add new server**
3. Select **Apache Tomcat**
4. Browse and select Tomcat installation folder
5. Click **Finish**

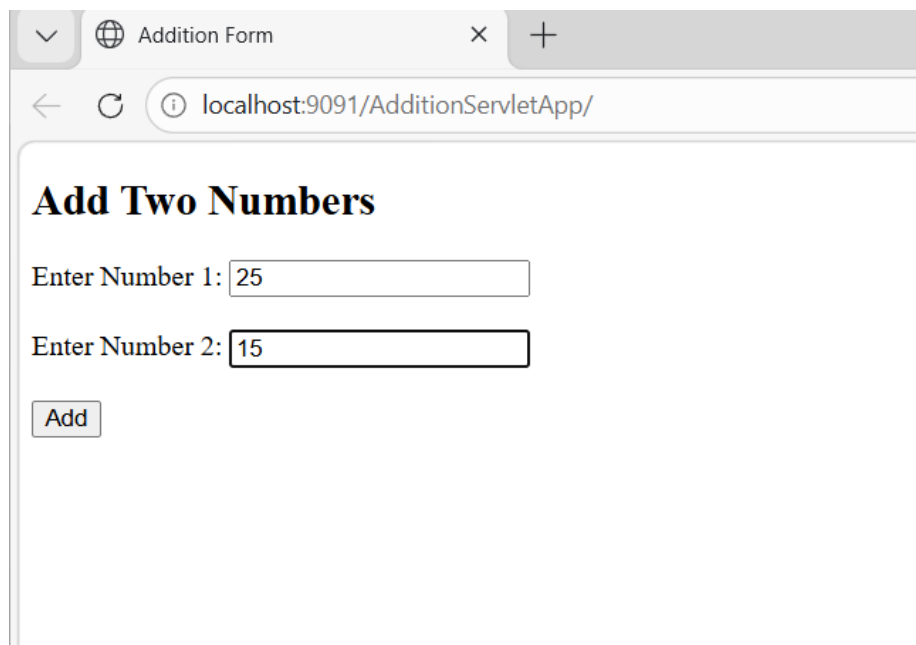
5. Run the Project

1. Right click project → **Run As** → **Run on Server**
2. Choose Tomcat → Finish

6. Output

Open browser:

<http://localhost:8080/AdditionServletApp/index.html>



The screenshot shows a web browser window with a single tab titled "Addition Form". The address bar displays "localhost:9091/AdditionServletApp/". The page content features the heading "Add Two Numbers" in bold. Below the heading, there are two input fields: "Enter Number 1:" with the value "25" and "Enter Number 2:" with the value "15". At the bottom left of the form is a button labeled "Add".

Q.2 Explain Servlet Life Cycle and demonstrate its methods with example.

Introduction to Servlet Life Cycle

- The **Servlet Life Cycle** defines the stages through which a servlet passes from its **creation to destruction**.
- It is **controlled by the web server** like Apache Tomcat.

2. Phases of Servlet Life Cycle

- A servlet has **three main life cycle methods**:

Phase 1: Loading and Instantiation

- The servlet class is **loaded into memory** by the server
- An **object of the servlet is created**
- This happens **only once**

Loading can be:

- Automatically (first request)
- At server startup

Phase 2: Initialization – init() Method

✓ Definition:

The init() method is called **only once** after the servlet object is created.

✓ Syntax:

```
public void init() throws ServletException
```

✓ Purpose:

- Initialize resources
- Setup configuration
- Establish database connection

✓ Key Points:

- Called only once in lifetime
- Cannot be called again and again

Phase 3: Request Processing – service() Method

✓ Definition:

The service() method is called **every time a client sends a request**.

✓ Syntax:

```
public void service(ServletRequest req, ServletResponse res)
```

✓ Working:

- It is the **heart of servlet processing**
- It decides which method to execute:
 - doGet() → for GET request
 - doPost() → for POST request

In real applications, we override:

- doGet() or doPost() instead of service()

Phase 4: Destruction – destroy() Method

✓ Definition:

The destroy() method is called **once before the servlet is removed from memory**.

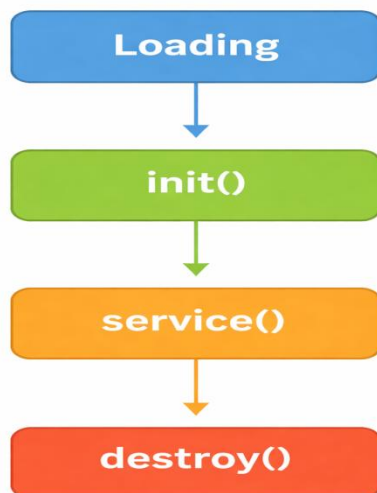
✓ Syntax:

```
public void destroy()
```

✓ Purpose:

- Release resources
- Close database connections
- Clean up memory

Life Cycle Diagram:



Detailed Flow

1. Servlet is loaded by server (Apache tomcat)
2. `init()` is called (once)
3. For each request:
 - `service()` executes
 - Calls `doGet()` / `doPost()`
4. When server stops:
 - `destroy()` is called

Example Program Demonstrating Life Cycle

```
package com.demo;

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/LifeCycleServlet")
public class LifeCycleServlet extends HttpServlet
{

    public void init()
    {
        System.out.println("Servlet Initialized");
    }

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws IOException
    {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<h2>Service Method Executed</h2>");
        out.close();
    }

    public void destroy()
    {
        System.out.println("Servlet Destroyed");
    }
}
```

Output:

Service Method Executed

Q.3 Design Java Application (using JSP) for registration of hackthon (student name, Mail Id, Phone No, Gender, Course) and store data in appropriate table.

1. Aim

To develop a **JSP-based Hackathon Registration System** and store student details in MySQL database.

2. Requirements

- Java JDK
- Eclipse IDE
- Apache Tomcat
- MySQL Workbench
- MySQL Connector/J

3. Database Creation (Using test DB)

Step 1: Open MySQL Workbench

Step 2: Use Database

USE test;

Step 3: Create Table

```
CREATE TABLE hackathon
(
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100),
    phone VARCHAR(15),
    gender VARCHAR(10),
    course VARCHAR(50)
);
```

4. Project Creation in Eclipse

Steps:

1. Open Eclipse
2. File → New → Dynamic Web Project
3. Project Name: HackathonRegistration
4. Select Apache Tomcat
5. Finish

5. Add JDBC Driver

- Copy mysql-connector-j-9.6.0.jar
- Paste into:

WEB-INF/lib

Step-by-Step Procedure:

Method 1: Add JAR to Build Path

1. Right click your project
2. Click:
Build Path → Configure Build Path
3. Go to:
Libraries tab - select ClassPath
4. Click:
Add External JARs
5. Select your file:

mysql-connector-j-9.6.0.jar

Click:

Apply → OK

✓ Now check:

Referenced Libraries

→ You should see your JAR file:mysql-connector-j-9.6.0.jar

Method 2:

Step 1:

- Right click on your project
- Example: HackathonRegistration

Step 2:

- Click on **Properties**

Step 3:

- In left side menu, select:
Deployment Assembly

Step 4:

- Click on **Add** button

Step 5:

- Select option: Java Build Path Entries

Click **Next**

Step 6:

- You will see list of libraries
- Tick the checkbox:

mysql-connector-j-9.6.0.jar

Step 7:

- Click **Finish**

Step 8:

- Click **Apply** → **OK**

6.JSP Form Page

STEP : Create 2 JSP Files

register.jsp
save.jsp

Right click project → New → JSP File

register.jsp

```
<!DOCTYPE html>
<html>
<body>
<h2>Hackathon Registration</h2>

<form action="save.jsp" method="post">
Name: <input type="text" name="name"><br><br>
Email: <input type="email" name="email"><br><br>
Phone: <input type="text" name="phone"><br><br>

Gender:
<input type="radio" name="gender" value="Male">Male
<input type="radio" name="gender" value="Female">Female<br><br>

Course:
<select name="course">
<option>MCA</option>
<option>BCA</option>
<option>BBA</option>
</select><br><br>

<input type="submit" value="Register">
</form>

</body>
</html>
```

7. JSP Database Code (IMPORTANT CHANGE)

save.jsp

```
<%@ page import="java.sql.*" %>

<%
```

```

String name = request.getParameter("name");
String email = request.getParameter("email");
String phone = request.getParameter("phone");
String gender = request.getParameter("gender");
String course = request.getParameter("course");

try
{
Class.forName("com.mysql.cj.jdbc.Driver");

Connection con = DriverManager.getConnection(
"jdbc:mysql://localhost:3306/test?useSSL=false&serverTimezone=UTC",
"root",
"root"
);

PreparedStatement ps = con.prepareStatement(
"INSERT INTO hackathon(name,email,phone,gender,course)
VALUES(?,?,?,?,?)"
);

ps.setString(1, name);
ps.setString(2, email);
ps.setString(3, phone);
ps.setString(4, gender);
ps.setString(5, course);

ps.executeUpdate();

out.println("<h2>Registration Successful!</h2>");

con.close();

} catch(Exception e) {
out.println(e);
}
%>

```

8. Execution Steps

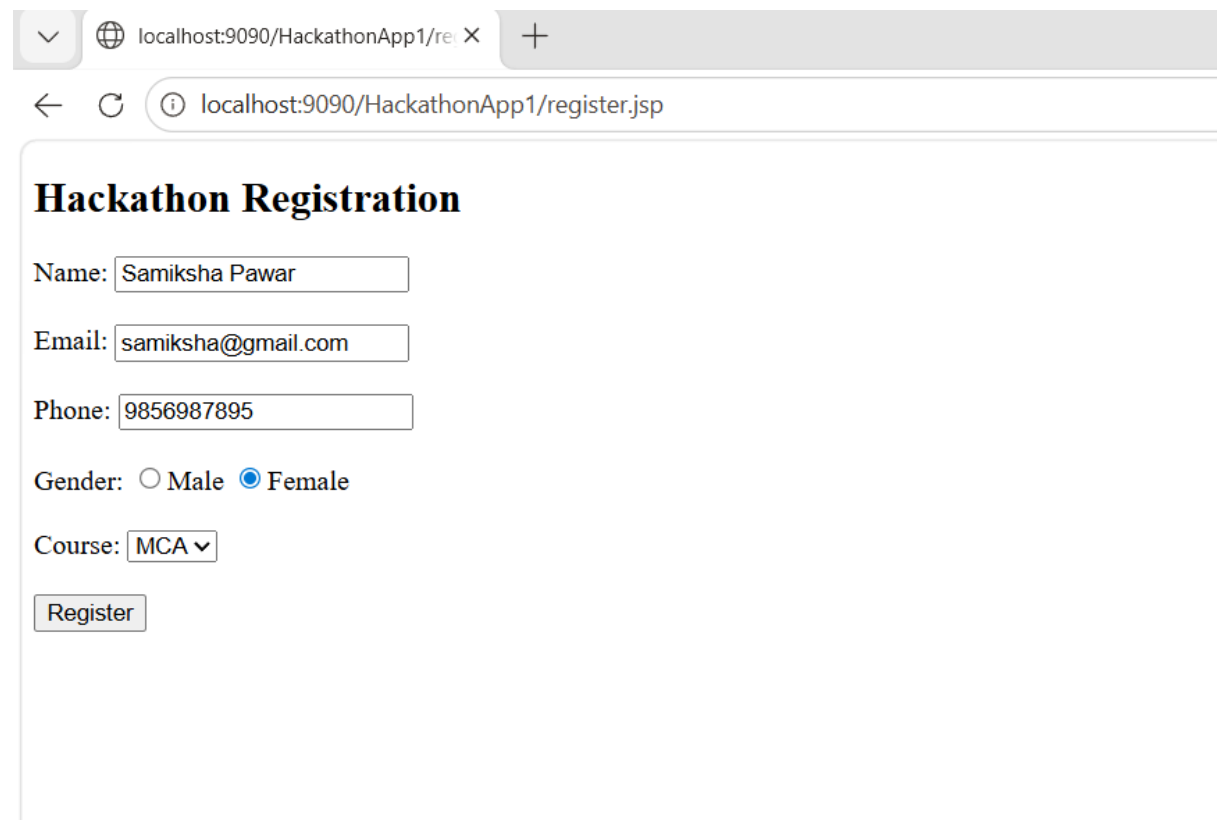
1. Right click project

2. Run on Server
3. Select Apache Tomcat

Open:

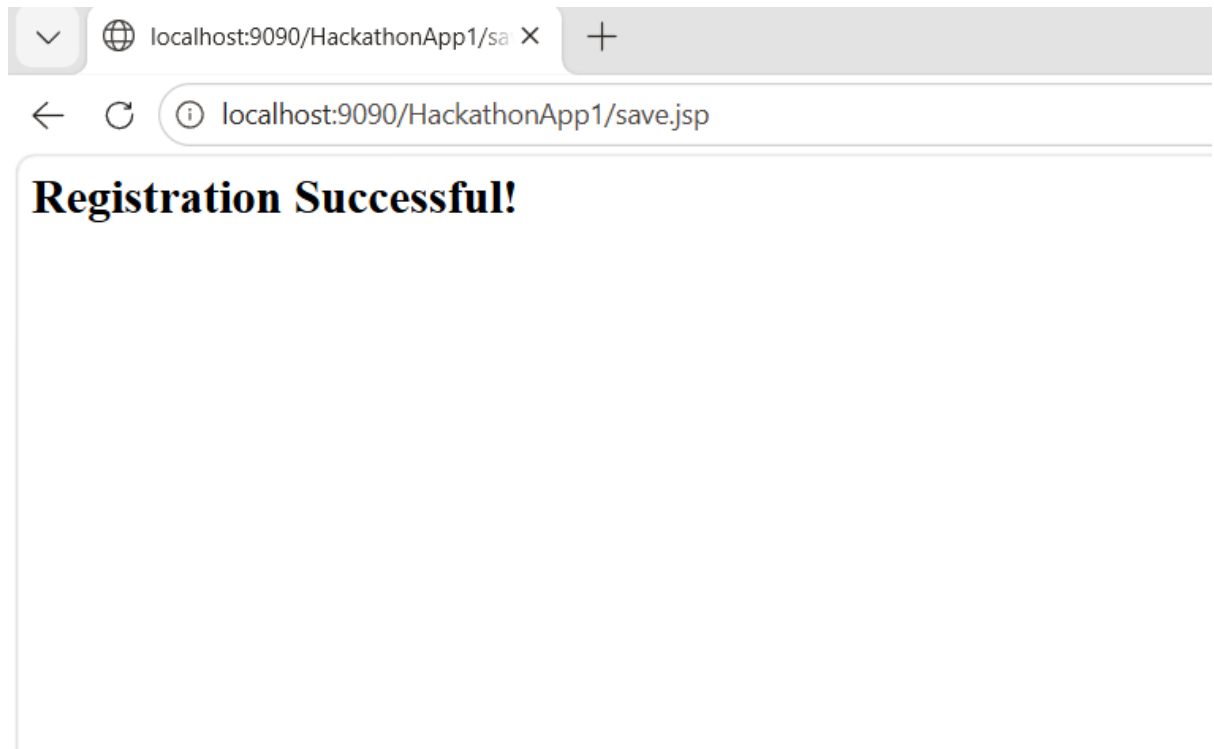
<http://localhost:8080/HackathonRegistration/register.jsp>

9. Output



The screenshot shows a web browser window with the address bar displaying `localhost:9090/HackathonApp1/register.jsp`. The page title is "Hackathon Registration". The form contains the following fields and controls:

- Name:
- Email:
- Phone:
- Gender: ☐ Male ☒ Female
- Course: (dropdown menu)
- Register:

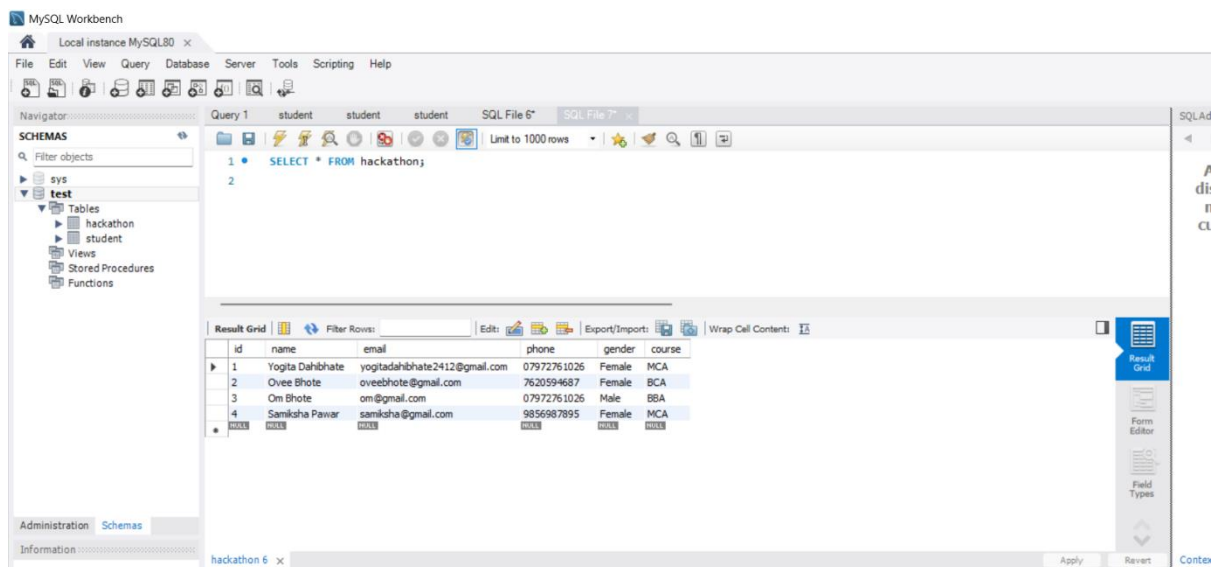


9. Verify Data in Database

Open MySQL Workbench

Run:

```
SELECT * FROM hackathon;
```



Q 4. Create a Java program to display student details (Name, Roll No, Marks) stored in a MySQL database using JDBC.

1. Aim

To create a Java (JSP) application that retrieves and displays student details (Roll No, Name, Marks) from MySQL database using JDBC.

2. Requirements

- Java JDK
- Eclipse IDE
- Apache Tomcat
- MySQL Workbench
- MySQL Connector/J

3. Database Design

Step 1: Use Database

USE test;

Step 2: Create Table (Student2)

```
CREATE TABLE student2
(
    rollno INT PRIMARY KEY,
    name VARCHAR(100),
    marks INT
);
```

Step 3: Insert Sample Data

```
INSERT INTO student VALUES
(1,'Sachin',85),
(2,'Ravi',78),
(3,'Amit',90);
```

4. Project Creation in Eclipse

Steps:

1. Open Eclipse
2. File → New → Dynamic Web Project
3. Project Name: **StudentApp**
4. Select Apache Tomcat
5. Click Finish

5. Add JDBC Driver

- Copy mysql-connector-j-9.6.0.jar
- Paste into:

WEB-INF/lib

Step-by-Step Procedure:

Method 1: Add JAR to Build Path

6. Right click your project
7. Click:
Build Path → Configure Build Path
8. Go to:
Libraries tab - select ClassPath
9. Click:
Add External JARs
10. Select your file:

mysql-connector-j-9.6.0.jar

Click:

Apply → OK

✓ Now check:

Referenced Libraries

→ You should see your JAR file:mysql-connector-j-9.6.0.jar

Method 2:

Step 1:

- Right click on your project
- Example: HackathonRegistration

Step 2:

- Click on **Properties**

Step 3:

- In left side menu, select:
Deployment Assembly

Step 4:

- Click on **Add** button

Step 5:

- Select option: Java Build Path Entries

Click **Next**

Step 6:

- You will see list of libraries
- Tick the checkbox:

mysql-connector-j-9.6.0.jar

Step 7:

- Click **Finish**

Step 8:

- Click **Apply** → **OK**

6. JSP Page (Display Student Data)

display.jsp

```
<%@ page import="java.sql.*" %>

<html>
<body>

<h2>Student Details</h2>

<table border="1">
<tr>
<th>Roll No</th>
<th>Name</th>
<th>Marks</th>
</tr>

<%
try
{
Class.forName("com.mysql.cj.jdbc.Driver");

Connection con = DriverManager.getConnection(
"jdbc:mysql://localhost:3306/test", "root", "root");

// □ Select Query
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM student2");

while(rs.next())
{
%>

<tr>
<td><%= rs.getInt("rollno") %></td>
<td><%= rs.getString("name") %></td>
<td><%= rs.getInt("marks") %></td>
</tr>

<%
}
}
```

```
con.close();  
  
}  
catch(Exception e)  
{  
out.println(e);  
}  
%>  
  
</table>  
  
</body>  
</html>
```

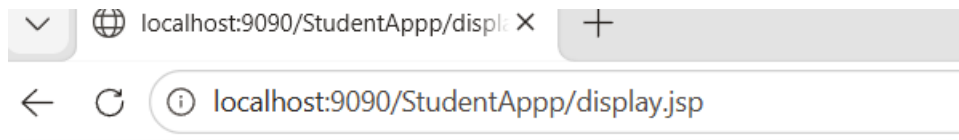
7. Execution Steps

1. Right click project → Run on Server
2. Select Apache Tomcat
3. Open browser:

<http://localhost:8080/StudentApp/display.jsp>

8. Output

- Displays all student records in **tabular format**
- Columns: Roll No, Name, Marks



Student Details

Roll No	Name	Marks
1	Sachin	85
2	Ravi	78
3	Amit	90

Q.5) Create a spring MVC form to read registration details for Blood donation camp with spring validation & display it.

STEP 1: Create Spring Boot Project

1. Open IntelliJ
2. Click **New Project**
3. Select **Spring Boot**
4. Fill:
 - Name: bloodcamp
 - Language: Java
 - Packaging: Jar
5. Add Dependencies:
 - ✓ Spring Web
 - ✓ Thymeleaf
 - ✓ Validation

Click **Finish**

STEP 2: Project Structure (Auto Created)

You will see:

src/main/java
src/main/resources

STEP 3: Create Model Class

1. Create Package

- Right click on com.blood.bloodcamp
- Click **New** → **Package**
- Name it: model

2. Create Java Class

- Right click on model
- Click **New** → **Java Class**
- Name: Donor

Code: Donor.java

```
package com.blood.bloodcamp.model;

import jakarta.validation.constraints.*;

public class Donor
{

    @NotEmpty(message="Name is required")
    private String name;

    @Email(message="Invalid email")
    private String email;

    @Min(value=18, message="Minimum age is 18")
    private int age;

    @NotEmpty(message="Blood group required")
    private String bloodGroup;

    // Getters and Setters
    public String getName()
    { return name; }
    public void setName(String name)
```

```

{ this.name = name; }

public String getEmail()
{ return email; }
public void setEmail(String email)
{ this.email = email; }

public int getAge()
{ return age; }
public void setAge(int age)
{ this.age = age; }

public String getBloodGroup()
{ return bloodGroup; }
public void setBloodGroup(String bloodGroup)
{ this.bloodGroup = bloodGroup; }
}

```

STEP 4: Create Controller

1. Create Package

- Right click on com.blood.bloodcamp
- Click **New** → **Package**
- Name it: controller

2. Create Java Class

- Right click on model
- Click **New** → **Java Class**
- Name: DonorController

Code: DonorController.java

```

package com.blood.bloodcamp.controller;

import com.blood.bloodcamp.model.Donor;
import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;

```

```

import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;

@Controller
public class DonorController {

    @GetMapping("/")
    public String showForm(Model model) {
        model.addAttribute("donor", new Donor());
        return "form";
    }

    @PostMapping("/submit")
    public String submitForm(@Valid @ModelAttribute("donor") Donor
donor,
                            BindingResult result) {

        if(result.hasErrors()) {
            return "form";
        }

        return "result";
    }
}

```

STEP 5: Create HTML Pages

Path: src/main/resources/templates

Code: form.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>

<h2>Blood Donation Form</h2>

<form th:action="@{/submit}" th:object="${donor}" method="post">

    Name: <input type="text" th:field="*{name}"/>
    <span th:errors="*{name}"></span><br><br>

```



```

Email: <input type="text" th:field="*{email}"/>
<span th:errors="*{email}"></span><br><br>

Age: <input type="text" th:field="*{age}"/>
<span th:errors="*{age}"></span><br><br>

Blood Group: <input type="text" th:field="*{bloodGroup}"/>
<span th:errors="*{bloodGroup}"></span><br><br>

<button type="submit">Submit</button>

</form>

</body>
</html>

```

Code: result.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>

<h2>Submitted Details</h2>

Name: <span th:text="${donor.name}"></span><br>
Email: <span th:text="${donor.email}"></span><br>
Age: <span th:text="${donor.age}"></span><br>
Blood Group: <span th:text="${donor.bloodGroup}"></span><br>

</body>
</html>

```

STEP 6: Run Project

1. Open main file:
BloodcampApplication.java
2. Click **Run** ►

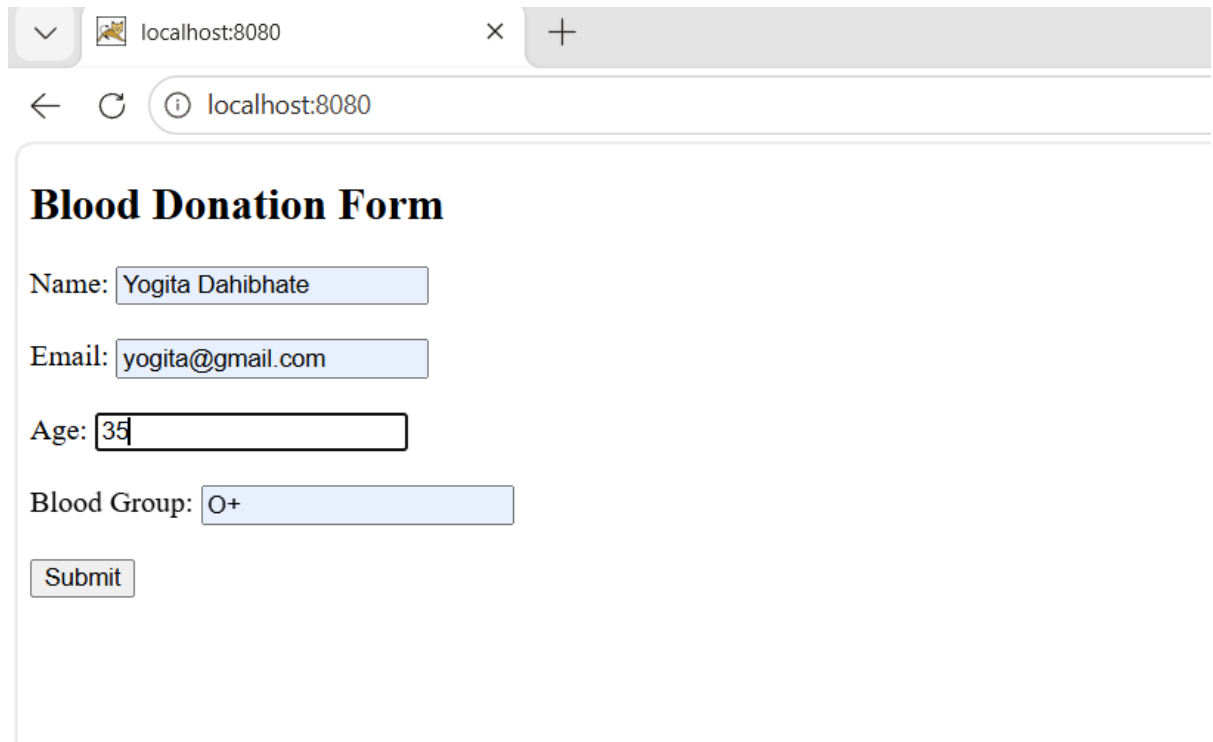
STEP 7: Open in Browser

Go to:

<http://localhost:8080/>

OUTPUT

- Form will open
- Enter data
- Validation works
- Data displayed on next page



The screenshot shows a web browser window with a single tab titled 'localhost:8080'. The address bar displays 'localhost:8080'. The page content features a form titled 'Blood Donation Form'. The form includes the following fields and values:

- Name:
- Email:
- Age:
- Blood Group:

At the bottom of the form is a 'Submit' button.

localhost:8080/submit

localhost:8080/submit

Submitted Details

Name: Yogita Dahibhate
Email: yogita@gmail.com
Age: 35
Blood Group: O+

Validation:

localhost:8080/submit

localhost:8080/submit

Blood Donation Form

Name: Name is required

Email: Invalid email

Age: Minimum age is 18

Blood Group:

